

实 习 汇 报

面向双栈虚拟机的 类 Python 脚本语言与交互式解释器 的设计与实现

报告人 · 徐 磊

实习单位 · 合肥元软软件有限公司

实习岗位 · 编译器开发

工程载体 · AtomicProof Compiler

汇报时间 · 2026 年 4 月

报告提纲

Outline

PART 一 实习情况 公司简介 实习岗位与工作内容	PART 二 论文进展 (重点) 论文题目 · 选题背景 课题内容 · 解决方案 技术难点 · 特色分析	PART 三 工作总结 阶段性成果 后续计划	PART 四 收获与建议 实习收获 意见与建议
--	---	--	---

本次汇报重点展开 第二部分 · 论文进展，涵盖选题、方案、难点与创新

一、实习情况 · 公司简介

合肥元软软件有限公司 · 元软（YuanRuan）

基本信息

企业名称	合肥元软软件有限公司
旗下品牌	元软（YuanRuan）
企业性质	技术密集型初创企业
核心赛道	Web3 应用 + 区块链底层设施
产品方向	区块链基础设施 · Web3 创新应用

业务定位与愿景

专注于 Web3 应用 与 区块链底层设施 研发，
致力于打造 改变世界的区块链基础设施，
探索 Web3 技术在全球化浪潮中的无限可能。

Web3

面向下一代去中心化应用

Infrastructure

底层基础设施研发

Innovation

多项独树一帜的产品

一、实习情况 · 实习工作介绍

实习岗位：编译器开发

岗位职责

- 参与公司自研区块链脚本语言的编译器研发
- 负责前端（词法 / 语法 / AST）模块迭代
- 参与中端 Visitor 分析与字节码后端实现
- 为编译器配套交互式调试器开发

技术栈

C++ 20	核心实现语言
CMake ≥ 3.28	跨平台构建系统
nlohmann/json	JSON 序列化与 ABI 输出
Bitcoin Script	目标双栈虚拟机指令集
MinGW-w64	Windows 交叉编译

实习期间主要工作产出

- 1) 完成约 800 KB C++ 编译器核心代码迭代 · 2) 设计并实现 四阶段 Pass + 四层 Visitor 架构
- 3) 实现 双栈调度原语 与 分层所有权系统 · 4) 搭建 BVM 模拟器 与 CLI 交互式调试器框架

二、论文进展 · 论文题目

Topic & Overview

学位论文题目（中文）

面向双栈虚拟机的类 Python 脚本语言与交互式解释器的设计与实现

Design and Implementation of a Python-like Scripting Language and Interactive Interpreter for Dual-Stack Virtual Machines

双栈虚拟机

类 Python 脚本

编译器

交互式调试器

比特币脚本

区块链合约

论文三大主线

① 语言层

类 Python 缩进语法
+ 静态类型与所有权

② 编译层

Pass 驱动 + 四层 Visitor
双栈字节码生成

③ 运行层

BVM 模拟器
+ CLI 交互式调试器

二、论文进展 · 选题背景与意义

Background & Motivation

区块链合约开发的三大痛点（双栈虚拟机视角）

①

原生脚本汇编化

比特币脚本只有约 100 个操作码，无变量、无函数；开发者必须以栈偏移而非变量名思考

②

抽象能力缺失

无结构化控制流块、无结构体、无库；复杂合约不可读也不可维护

③

调试手段匮乏

只有栈而无寄存器，传统调试模型无法直接套用；链上失败仅得到布尔结果

论文意义

理论价值：填补「双栈 VM 上的高级语言编译模型」与「栈槽粒度所有权系统」的研究空白

工程价值：为 UTXO 型区块链提供可读的高级语言与端到端工具链

实用价值：把「盲写 + 链上试错」升级为可断点、可检视、可单步的工程化开发流程

二、论文进展 · 课题内容

Research Goals & Contents

研究目标

- G1** 语法接近 Python，类型系统支持静态标注与线性所有权
- G2** 字节码长度较朴素方案节省 $\geq 30\%$
- G3** 编译期静态识别所有权 / 类型 / 栈不平衡错误
- G4** 提供可断点、可检视、可单步的交互式调试器
- G5** 10+ 代表性合约的基准评测与正确性验证

研究内容

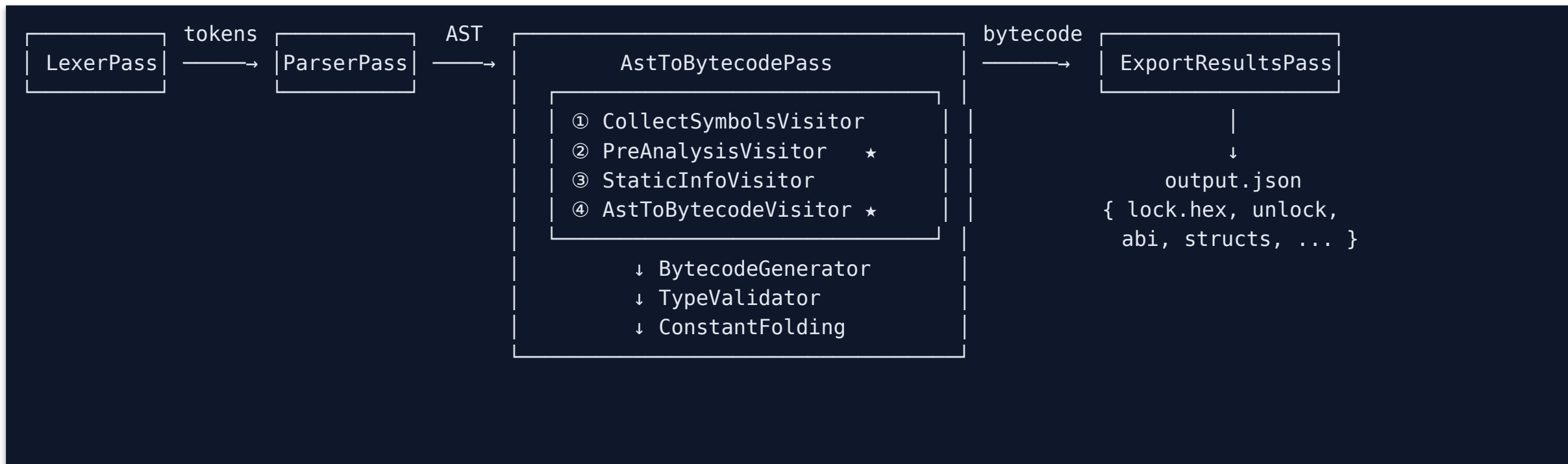
- R1** 类 Python 脚本语言的语法与类型系统设计
- R2** Pass 驱动的多阶段编译器架构与四层 Visitor
- R3** 双栈编译模型与分层所有权系统
- R4** BVM 模拟器与 CLI 交互式调试器
- R5** 评测基准建设与定量对比分析

拟解决的关键问题

- K1 Python 缩进语法与静态类型 / 线性所有权如何统一？
- K2 栈槽粒度的 use-after-consume 如何静态验证？
- K3 控制流分支后的双栈平衡如何保证？
- K4 「变量 = 标签」的零成本重命名何时必须插入 OP_ROLL？

二、论文进展 · 解决方案（一）总体架构

Pass-driven Compilation Pipeline



架构要点

- 四阶段 Pass —— 基于 PassManager 的依赖注入与拓扑排序，可独立启用 / 禁用
- 四层 Visitor —— 符号收集 → 预分析 → 静态信息 → 字节码生成；前后互为镜像，错误前置
- 三件套后端 —— BytecodeGenerator + Scope（栈槽符号表）+ TypeValidator（类型校验）
- 统一输出 —— output.json 含锁定脚本 / 解锁模板 / ABI / 结构体定义

二、论文进展 · 解决方案（二）语言设计与前端

Python-like Syntax · Lexer & Parser

类 Python 脚本示例

```
Contract P2PKH:                                # 合约容器
    def verify(sig: hex,                        # 静态类型标注
               pubKey: hex):
        pubKey_copy = pubKey.Clone()
        pubKeyHash  = Hash160(pubKey_copy)
        EqualVerify(pubKeyHash,
                     self.pubKeyHash)          # 成员变量
        result = CheckSig(sig, pubKey)
```

前端实现要点

模块	关键设计
Lexer	缩进栈 → INDENT/DEDENT 替代 {}
Lexer	53 种 TokenType + 比特币地址识别
Parser	手写递归下降，错误恢复直观
AST	Contract / Function / Struct 等节点

核心语言特性

Contract / Library
顶层容器 + import 复用

Struct（嵌套）
结构体 + 定长数组字段

for in Range(...)
受限循环（编译期可展开）

if / else
分支 + --asa 副栈控制

Delete / Clone / move
三元所有权原语

二、论文进展 · 解决方案（三） 四层 Visitor 与所有权

Semantic Analysis Pipeline

#	Visitor	职责	关键数据结构
①	CollectSymbolsVisitor	收集函数 / 结构体 / 库定义；唯一性检查	m_definedFunctions / m_structDefinitions
②	PreAnalysisVisitor ★	所有权追踪 · 消耗性操作合法性 · 类型检查	VariableInfo + 所有权位图
③	StaticInfoVisitor	产出 ABI / 构造器参数 / 结构体定义	m_abijson / m_structjson
④	AstToBytecodeVisitor ★	遍历 AST 生成字节码；操纵符号表与 Scope	Scope / SymbolTable

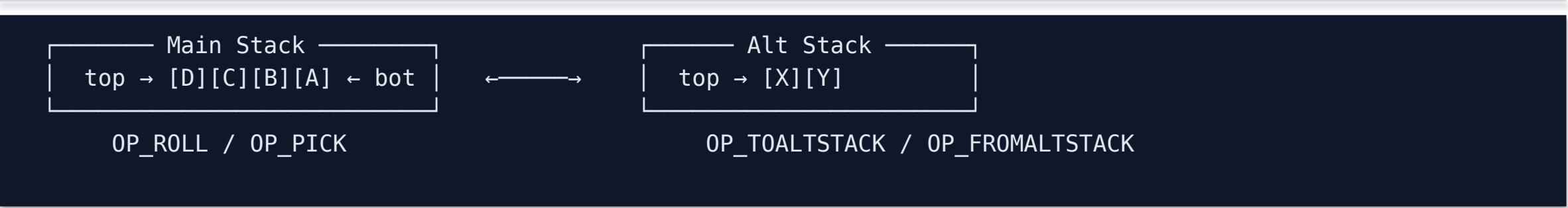
分层所有权位图 · 栈槽粒度的静态验证

```
struct VariableInfo {
    DataSource      source; // CONTRACT_MEMBER / CONSTANT / STACK /
    BUILTIN
    VariableState   state;  // DECLARED → USED → CONSUMED
    std::vector<bool> elementOwnership; // 元素级位图
    std::map<std::string,
            VariableState> fieldOwnership; // 字段级
};
```

```
# 部分消耗合法
numbers: int[4] = [...]
a = move(numbers[0])    # 元素 0 → CONSUMED
b = numbers[1].Clone()  # 元素 1 仍 USED
c = move(numbers[2])    # 元素 2 → CONSUMED
# numbers[1], numbers[3] 仍可用
```

二、论文进展 · 解决方案（四）双栈模型与零成本重命名

Dual-Stack VM + Zero-cost Renaming



双栈高层原语

原语	语义	字节码
SetAlt(x)	主栈 → 副栈	OP_TOALTSTACK
SetMain(x)	副栈 → 主栈	OP_FROMALTSTACK
Keep(x)	原子保留副本	OP_DUP+OP_TOALTSTACK

零成本重命名 · 字节码节省

```
first = numbers[0]    # 0 字节码
second = numbers[1]   # 0 字节码
return first + second # 1 字节码
```

传统: 52 79 51 79 52 7a 52 7a 93 (9B)
本方案: 52 7a 52 7a 93 (5B -44%)

核心洞察

变量名只是栈位置的标签；赋值不复制数据，只重命名标签。
在代表性合约上字节码体积平均下降 ~44%，直接对应链上手续费降低。

二、论文进展 · 解决方案（五）交互式解释器

BVM Simulator + CLI Debugger

BVM 模拟器 · 执行核心

```
class BVMSimulator {
    stack<StackItem>  mainStack, altStack;
    vector<CallFrame> callStack;
    size_t             pc;
    ExecutionState     state;    // READY/RUNNING
                                // PAUSED/FINISHED/ERROR

    void step();              // 单步
    void run();              // 直至断点
    void onBreakpoint(Breakpoint&);
};
```

CLI 调试器 · 会话样例

```
$ apc --debug test/p2pkh.ct
(apc) break verify:12
(apc) run
[paused] p2pkh.ct:12  EqualVerify(...)
(apc) print pubKeyHash
    → 0x89abcdef...
(apc) stack
(apc) step-over
(apc) backtrace
(apc) continue
```

调试器六大子模块

core/ DebuggerCore 编译入口 + 调试信息加载	vm/ BVMSimulator 双栈执行模拟	info/ DebugInfo 源码 ↔ PC ↔ 变量
breakpoint/ BPManager 行 / 条件 / 命中计数	inspector/ Inspector 变量还原 / 表达式求值	interface/ CliDebugger REPL 命令循环

二、论文进展 · 技术难点

Key Technical Challenges

D1

栈槽粒度的所有权静态验证

栈式 VM 中操作消耗栈顶元素，必须在编译期识别 use-after-consume ；
需在 数组级 / 元素级 / 字段级 三个层级上独立追踪状态，且组合规则复杂。

→ 采用 三态状态机 + 多层位图，构成 **PreAnalysisVisitor** 的核心

D2

结构化控制流到双栈的映射

if / else / for 等分支后必须保持 主栈 + 副栈 的双重平衡；
副栈在子作用域使用易破坏全局栈不变量。

→ 默认禁用子作用域 **SetAlt**（--asa 显式开启），形式化证明栈平衡性

D3

「变量 = 标签」与栈顶执行的协调

栈顶先出与命名访问的语义鸿沟：何时必须 OP_ROLL，何时仅重命名？
误判将导致字节码膨胀或栈错位。

→ **Scope::getPos()** + **roll()** 仅在跨位置时插入指令，否则零成本重命名

D4

双栈 VM 的源码级调试

VM 仅可观测指令流；要在断点处还原源码变量与表达式值，
需建立 源码行 ↔ PC ↔ 变量名 ↔ 栈位置 四元映射。

→ 字节码生成同步产出 **DebugInfo**，配合 **Inspector + Evaluator** 实现

二、论文进展 · 特色与创新

Features & Innovations

研究特色

- 端到端工具链 —— 语言、编译器、虚拟机、调试器一体化
- 面向真实 VM —— 比特币脚本标准，结果可直接上链验证
- 理论与工程并重 —— 形式化规则 + 可复现开源实现

四大创新点

C1	面向栈槽的分层所有权 首次将线性类型细化到 栈元素粒度
C2	零成本重命名模型 字节码节省 ~44%
C3	双栈高层原语 SetAlt / SetMain / Keep 抽象
C4	双栈 VM 上的源码级调试器 填补该领域工具空白

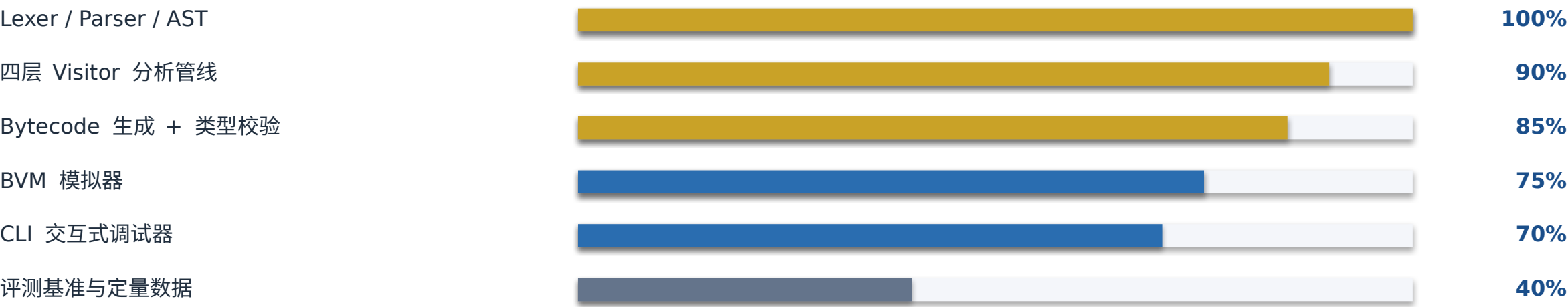
已达成的量化指标

~44% 字节码节省率	4 层 Visitor	100+ 内置函数 / 操作码	3 平台 Linux / macOS / Windows	10+ 基准合约
-----------------------	-----------------------	---------------------------	--	--------------------

二、论文进展 · 当前进度与后续计划

Progress & Roadmap

当前进度（截至 2026.04）



后续 14 个月时间轴

2026.05-07	理论补齐	EBNF / 类型规则 / 所有权状态机形式化
2026.08-10	编译器打磨	PreAnalysis 重构 + 字节码优化
2026.11-12	调试器 + 评测	调试器 v1.0 + 定量数据
2027.01-02	论文撰写	全稿 ≥ 6 万字
2027.03-05	评审 + 答辩	中期 → 预答辩 → 答辩

三、实习收获与建议

Takeaways & Suggestions

实习收获

技术能力

深入掌握 C++20 现代特性、CMake 工程化、编译器架构与栈式虚拟机执行模型

工程素养

代码评审、跨平台构建、错误管理、单元测试、Git 协作流程

领域知识

比特币脚本规范、UTXO 模型、Web3 / 区块链底层基础设施

研究能力

把工程问题抽象为论文研究问题，形式化描述 + 定量评测

建议与展望

对自己

持续打磨编译器原型 → 论文成果；关注国际同类项目的最新进展

对学校

鼓励学生进入有真实场景的初创企业；推动「论文研究 ↔ 工程产出」双向赋能

对企业

建议建立内部技术分享与代码评审制度，持续输出技术博客以提升行业影响力

对行业

Web3 与编译技术的交叉是蓝海方向；双栈 VM 工具链有广阔产学研应用空间

感谢合肥元软软件有限公司提供的实习机会 · 感谢导师与同事的悉心指导 · 敬请批评指正